

Rohit Raj Singh day-14

1. Input Validation

Input validation means checking whether the data received from a client is correct, complete, and follows the required rules before processing it.

For example, an Employee API can validate:

- Name is required.
- Email must have a valid email format.
- Salary must be greater than 0.
- Department must be valid.
- String fields must satisfy length requirements.

Example:

```
class EmployeeCreate(BaseModel):  
  
    name: str = Field(min_length=2, max_length=50)  
  
    email: EmailStr  
  
    salary: float = Field(gt=0)
```

If the client sends an invalid email, FastAPI/Pydantic rejects the request instead of sending invalid data to the database.

Purpose: Prevent invalid or unsafe data from entering the application.

2. Error Responses

An **error response** tells the client that a request could not be processed and explains what went wrong.

A good API should return a **clear and consistent error format**.

Example:

```
{  
  "success": false,  
  "message": "Employee not found",  
  "error_code": "EMPLOYEE_NOT_FOUND"  
}
```

Common API errors include:

- Invalid input
- Employee not found
- Duplicate email
- Database constraint violation

- Unexpected server error

Purpose: Make errors understandable and predictable for API clients.

3. HTTP Status Codes

HTTP status codes communicate the result of an HTTP request.

Common status codes used in APIs:

Status Code	Meaning	Example
200	OK	Employee fetched successfully
201	Created	Employee created
400	Bad Request	Invalid request/business input
404	Not Found	Employee does not exist
409	Conflict	Duplicate email
422	Unprocessable Entity	Validation error
500	Internal Server Error	Unexpected server error

Example:

```
raise HTTPException(status_code=404, detail="Employee not found")
```

Purpose: Allow clients to understand the result of their request programmatically.

4. Pagination

Pagination means dividing a large amount of data into smaller pages instead of returning all records at once.

Example:

```
GET /employees?page=1&limit=20
```

Here:

- `page=1` → first page
- `limit=20` → maximum 20 employees per page

For example:

```
Page 1 → Employees 1-20  
Page 2 → Employees 21-40  
Page 3 → Employees 41-60
```

Purpose: Reduce response size, improve performance, and make large datasets easier to handle.

5. Filtering

Filtering means returning only the records that match specific conditions.

Example:

```
GET /employees?department=IT
```

This returns only employees whose department is **IT**.

Another example:

```
GET /employees?search=John
```

This can return employees whose names contain **John**.

Example SQLAlchemy condition:

```
select(Employee).where(Employee.department == department)
```

Purpose: Allow clients to retrieve only the data they need.

6. Sorting

Sorting means arranging API results according to a specific field or order.

Example:

```
GET /employees?sort=name
```

This can return employees alphabetically by name.

Other possible sorting fields:

```
sort=name  
sort=salary  
sort=department
```

Example SQLAlchemy:

```
select(Employee).order_by(Employee.name)
```

Purpose: Make API results easier to read and allow clients to control the order of returned data.

7. API Consistency

API consistency means following the same structure and behavior across different endpoints.

For example, all successful responses can follow a consistent structure:

```
{  
    "msg": "data fetched",  
    "data": [...]  
}
```

Similarly, errors should follow a consistent format:

```
{  
    "success": false,
```

```
    "message": "Employee not found",
    "error_code": "EMPLOYEE_NOT_FOUND"
}
```

Consistency should be maintained in:

- Response structure
- Error format
- Naming conventions
- HTTP status codes
- Query parameter behavior

Purpose: Make the API predictable and easier for frontend developers and other clients to use.

8. Logging

Logging means recording important events, requests, errors, and application activities so developers can monitor and troubleshoot the API.

Python provides the `logging` module:

```
import logging
```

```
logger = logging.getLogger(__name__)
```

```
logger.info("Employee created successfully")
```

```
logger.error("Failed to create employee")
```

Common logging levels:

Level	Purpose
DEBUG	Detailed debugging information
INFO	Normal application events
WARNING	Something unexpected but not necessarily an error
ERROR	An operation failed
CRITICAL	Serious application failure

Purpose: Help developers monitor the application and diagnose problems without exposing internal details to API users.

9. Exception Handling

Exception handling is the process of catching and managing errors that occur while the application is running.

Example:

```
try:
```

```
    db.commit()
```

```
except IntegrityError:
```

```
    db.rollback()
```

```
    raise
```

In FastAPI, expected API errors can also be returned using:

```
raise HTTPException(
```

```
    status_code=404,
```

```
    detail="Employee not found"
```

```
)
```

Exception handling prevents unexpected errors from crashing the application and allows the API to return meaningful responses.

Common Flow

Request



Validation



Business Logic



Database Operation



Exception occurs



Exception Handler



Consistent Error Response

Purpose: Handle failures safely and provide useful responses to API clients.

Validation Documentation

The Employee Management API uses **Pydantic validation** to validate incoming request data before processing it.

Validation Rules

Field	Validation
name	Required, minimum 2 and maximum 50 characters
email	Must be a valid email address
department	Minimum 2 and maximum 50 characters
designation	Minimum 2 and maximum 50 characters
salary	Must be greater than 0
is_active	Boolean value
page	Must be greater than or equal to 1
limit	Between 1 and 100
order	Supports ascending (<code>asc</code>) and descending (<code>desc</code>) sorting

Invalid request data is automatically rejected by FastAPI/Pydantic with a **422 Unprocessable Entity** response.

Error Response Examples

The API uses HTTP status codes to indicate different types of errors.

1. Employee Not Found — 404

```
{
  "detail": {
    "msg": "Employee not found"
  }
}
```

2. Invalid Email — 422

```
{
  "detail": [
    {
      "loc": ["body", "email"],
      "msg": "value is not a valid email address"
    }
  ]
}
```

3. Duplicate Email — 409

```
{
  "detail": "Email already exists"
}
```

4. Invalid Salary — 422

Example:

```
{
  "salary": -5000
}
```

The request is rejected because salary must be greater than 0.

5. Internal Server Error — 500

Used when an unexpected error occurs on the server while processing the request.

```
{
  "detail": "Internal server error"
}
```

6. Bad Request — 400

Used when the request is syntactically valid but cannot be processed because of invalid data or a database constraint.

```
{
  "detail": "Database constraint violated"
}
```

Test Cases

The following test cases are used to verify the functionality and validation of the Employee Management API.

Test Case	Request / Condition	Expected Result
Create Employee	POST /employees with valid data	201 Created
Get Employees	GET /employees	200 OK
Pagination	GET /employees?page=1&limit=2	Returns limited records
Search	GET /employees?search=Rohit	Returns matching employees
Sorting ASC	GET /employees? sort=name&order=asc	Employees sorted A → Z
Sorting DESC	GET /employees? sort=name&order=desc	Employees sorted Z → A
Get Employee	GET /employees/1	Returns employee details

Test Case	Request / Condition	Expected Result
Update Employee	PUT /employees/1	Employee updated
Partial Update	PATCH /employees/1	Selected fields updated
Delete Employee	DELETE /employees/1	Employee deleted
Invalid Email	Invalid email format	422 Unprocessable Entity
Invalid Salary	Salary $\leq$ 0	422 Unprocessable Entity
Missing Employee	Non-existing employee ID	404 Not Found
Duplicate Email	Existing email	409 Conflict